2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
October 14-18, 2024. Abu Dhabi, UAE

MM3DGS SLAM: Multi-modal 3D Gaussian Splatting for SLAM Using Vision, Depth, and Inertial Measurements

Lisong C. Sun*, Neel P. Bhatt*†, *Member, IEEE*, Jonathan C. Liu, Zhiwen Fan, Zhangyang Wang, *Senior Member, IEEE*, Todd E. Humphreys, *Senior Member, IEEE*, and Ufuk Topcu, *Fellow, IEEE*

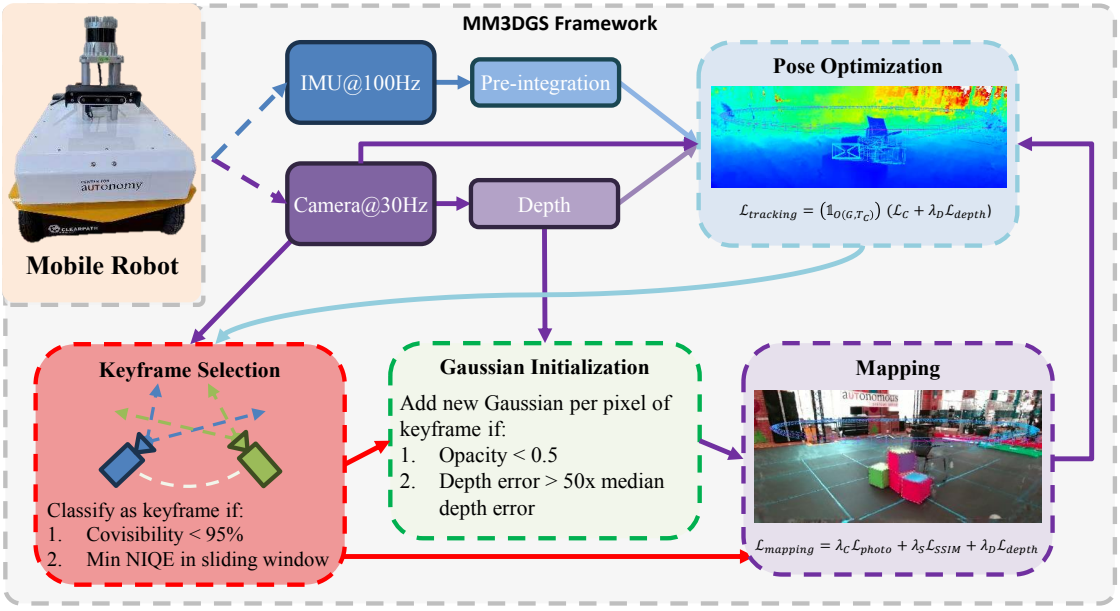

Fig. 1: Overview of the MM3DGS framework. We receive camera images and inertial measurements from a mobile robot. We utilize depth measurements and IMU pre-integration for pose optimization using a combined tracking loss. We apply a keyframe selection approach based on image covisibility and the NIQE metric across a sliding window and initialize new 3D Gaussians for keyframes with low opacity and high depth error [1]. Finally, we optimize parameters of the 3D Gaussians according to the mapping loss for the selected keyframes.

***Abstract*— Simultaneous localization and mapping is essential for position tracking and scene understanding. 3D Gaussian-based map representations enable photorealistic reconstruction and real-time rendering of scenes using multiple posed cameras. We show for the first time that using 3D Gaussians for map representation with unposed camera images and inertial measurements can enable accurate SLAM. Our method, MM3DGS, addresses the limitations of prior neural radiance field-based representations by enabling faster rendering, scale awareness, and improved trajectory tracking. Our framework enables keyframe-based mapping and tracking utilizing loss functions that incorporate relative pose transformations from pre-integrated inertial measurements, depth estimates, and measures of photometric rendering quality. We also release a multi-modal dataset, UT-MM, collected from a mobile robot equipped with a camera and an inertial measurement unit. Experimental evaluation on several scenes from the dataset shows that MM3DGS achieves nearly 3x improvement in tracking and 5% improvement in photometric rendering quality compared to the current 3DGS SLAM state-of-the-art, while allowing real-time rendering of a high-resolution dense 3D map.**

All authors are with the University of Texas at Austin, Austin, TX, USA. *Equal contribution and co-first authors; †Corresponding author; email: npbhatt@utexas.edu. This work was supported by the Defense Advanced Research Projects Agency (DARPA) contract FA8750-23-C-1018. Approved for Public Release, Distribution Unlimited.

I. INTRODUCTION

Simultaneous localization and mapping (SLAM), the task of generating a map of the environment along with estimating the pose of a sensor, is an essential enabler in applications such as aerial mapping, augmented reality, and autonomous mobile robotics [2], [3], [4]. 3D scene reconstruction and localization are essential capabilities for an autonomous system to perform downstream tasks such as decision-making and navigation [5]. As a result, SLAM plays a pivotal role in advancing the capabilities of autonomous systems.

The representation used for mapping and the type of sensor input utilized is a fundamental choice that has a direct impact on SLAM performance. Although SLAM approaches using sparse point clouds to represent the operating environment yield state-of-the-art tracking accuracy [6], [7], the generated maps are disjoint due to sparsity and visually inferior to newer 3D reconstruction methods. While visual quality is irrelevant for the sole purpose of navigation, the creation of photorealistic maps is valuable for human consumption, semantic segmentation, and post-processing. Neural radiance

Webpage: https://vita-group.github.io/MM3DGS-SLAM

field (NeRF)-based SLAM approaches yield output maps that are spatially and photorealistically dense [8], [9], but are computationally expensive at inference time and hence are not capable of accurately tracking in real-time. To address this shortcoming, recent works utilize the real-time rendering capability of a 3D Gaussian-based map representation and perform 3D Gaussian splatting (3DGS) to generate 2D renderings [10], [11], [12].

A precursor to 3DGS is depth initialization of the Gaussians. Most approaches use depth inputs from relatively expensive sensors such as LiDARs, which may not be readily available on a device such as a consumer phone [13]. Other approaches use an RGB-D camera providing depth information through stereoscopic depth estimation [11]. However, relying solely on RGB-D cameras may lead to erroneous depth, especially as the distance from the camera increases, leading to degraded tracking accuracy.

To address these shortcomings, we present the first real-time visual-inertial SLAM framework using 3D Gaussians for efficient and explicit map representation with inputs from a single monocular camera or RGB-D camera along with inertial measurements that may be readily obtained from most modern smartphones. Our approach is capable of real-time photorealistic 3D reconstructions of the environment, yielding accurate camera pose tracking for SLAM. We release a multi-modal dataset consisting of several scenes and the required sensory inputs which we use to evaluate our approach. Our approach outperforms SplaTAM, the state-of-the-art RGB-D-based 3DGS SLAM approach, by nearly **3x** in trajectory tracking and achieves a **5**% increase in photorealistic rendering quality.

In summary, our key contributions are as follows:

- We integrate inertial measurements and depth estimates from an unposed monocular RGB or RGB-D camera into our real-time MM3DGS SLAM framework using 3D Gaussians for scene representation. Our framework enables scale awareness as well as lateral, longitudinal, and vertical trajectory alignment. Our framework can utilize inputs from inexpensive sensors available on most consumer smartphones.
- We release a multi-modal dataset, dubbed UT-MM, collected using a mobile robot consisting of several indoor scenarios with RGB and RGB-D images, LiDAR depth, 6-DOF inertial measurement unit (IMU) measurements, as well as ground truth trajectories for error analysis.
- We achieve superior quantitative and qualitative trajectory tracking (nearly **3x** improvement) and photometric rendering (**5%** improvement) results compared to the current state-of-the-art 3DGS SLAM baseline.

II. RELATED WORKS

A. SLAM Map Representations

Sparse visual SLAM algorithms, such as ORB-SLAM [6] and OKVIS [7], are designed to estimate precise camera poses while producing only sparse point clouds for map representation. Conversely, dense visual SLAM methodologies [14], [15] aim to construct a dense representation of the scene. Map representations in dense SLAM are broadly classified into two categories: view-centric and world-centric. View-centric representations encode 3D information using keyframes accompanied by depth maps [14]. On the other hand, world-centric approaches anchor the 3D geometry of the entire scene within a consistent global coordinate system, typically represented through surfels [15] or by utilizing occupancy values within voxel grids [16].

B. Efficient 3D Representation

Recent advancements in neural rendering, exemplified by NeRFs [17] and subsequent developments [18], [19], have demonstrated significant progress in novel view synthesis using foundational 3D representations such as MLPs, voxel grids, or hash tables. Although these NeRF-inspired models exhibit impressive results, they often necessitate extensive training periods for individual scenes. The introduction of 3DGS, which this paper employs, offers a solution to the issue of training efficiency. This volumetric rendering approach depicts a 3D scene as a collection of explicit Gaussian distributions [20]. Initially, reconstructing a scene with 3DGS required applying structure-from-motion techniques like COLMAP [21], [22] to determine camera poses before optimization, but recent studies have aimed to bypass this prerequisite by leveraging depth measurements or monocular depth estimators [11], [23]. Concurrently, several research initiatives have explored the utilization of 3DGS within SLAM frameworks, with works in [24], [11], [10], [25] employing RGB-D sequences. Specifically, Gaussian Splatting SLAM [10] considers both RGB and RGB-D settings, but none of these frameworks fuse inertial measurements. At the time of writing, SplaTAM, an RGB-D-based 3DGS method, is the only method with publicly available source code and is thus considered as the baseline [11].

C. Multi-modal SLAM Frameworks

Visual sensing, while effective, has its limitations, such as susceptibility to motion blur and exposure changes. To mitigate the vulnerabilities of individual sensors, a standard approach is to employ multi-modal sensing through sensor fusion. Prior research has delved into enhancing system robustness by integrating semantic mapping and developing LiDAR-camera SLAM systems augmented with laser range finders [13], [26], [27]. In contrast, our focus is on the fusion of inertial measurements, which are favored for their high data acquisition rates and proficiency in tracking rapid movements within brief timeframes.

III. METHOD

The MM3DGS SLAM framework consists of four main stages: pose optimization (tracking), keyframe selection, Gaussian initialization, and mapping. An overview of the framework is illustrated in Fig. 1.

A. 3D Gaussian Splatting

The underlying scene map is represented using a set of 3D Gaussians $\mathcal{G}$, where the ith Gaussian is defined by position

$\boldsymbol{\mu}_i$, shape Σ_i, opacity o_i, and color $\boldsymbol{c}_i$. Recall that given a mean, $\boldsymbol{\mu}$, and covariance matrix, Σ, a Gaussian distribution is defined as

$$G(\boldsymbol{x}) = \exp(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}) \tag{1}$$

By applying eigenvalue decomposition to Σ, the covariance can be decomposed into the form, $\Sigma = RSS^{\mathsf{T}}R^{\mathsf{T}}$, where R represents an orthonormal rotation matrix and S represents a diagonal scaling matrix. In this way, the shape of 3D Gaussians can be optimized while keeping Σ symmetrical and positive-semidefinite.

A set of 3D Gaussians can be rasterized into an image via "splatting," i.e., projecting the Gaussians onto a 2D image plane. The 2D view-space covariance matrix, Σ', can be computed as $\Sigma' = JW\Sigma(JW)^{\mathsf{T}}$, where J is the Jacobian of the affine approximation of the projection matrix, W is the world to view frame transformation matrix, and Σ is the 3D covariance matrix of the respective Gaussian.

Specifically, given a set of Gaussian features, $\mathcal{G}$, and a camera pose, T_c, the color, C, of a pixel is calculated by blending N Gaussians that overlap the pixel, ordered by non-increasing depth given by

$$C(\mathcal{G}, T_c) = \sum_{i=1}^{N} \boldsymbol{c}_i \alpha_i \prod_{j=1}^{i-1}(1-\alpha_j) \tag{2}$$

where $\boldsymbol{c}_i$ is the color of the ith Gaussian and α_i is sampled from the ith splatted 2D Gaussian distribution at the pixel location. Similarly, the pixel opacity can be calculated as

$$O(\mathcal{G}, T_c) = \sum_{i=1}^{N} \alpha_i \prod_{j=1}^{i-1}(1-\alpha_j) \tag{3}$$

Since this rendering process is differentiable, $\mathcal{G}$ can be optimized using the L_1 loss between the rendered image and the ground truth undistorted image, I:

$$\mathcal{L}_{\text{photo}} = L_1(I, C(\mathcal{G}, T_c)) \tag{4}$$

B. Depth Supervision

When tracking camera poses using a 3D Gaussian map, it is essential for the map to encode accurate geometric information, the absence of which may lead to divergence. This is particularly true for an underoptimized map, where Gaussians are not trained long enough to converge to the correct position. The use of depth priors solves this problem by providing reasonable initial position estimates, minimizing both inconsistent geometry and training time. Further, depth priors can supervise the map training loss to prevent geometric artifacts from overfitting on limited views.

To render depth in a differentiable manner, a second rasterization pass is performed with color $\boldsymbol{c}_i$ of each Gaussian replaced with its projected depth on the image plane.

In the RGB-D case, these depth priors can be gathered directly through a depth sensor or using stereoscopic depth. However, in the absence of a such sensors, monocular dense depth estimation networks, such as DPT [28], can be used. Since dense depth estimators output a relative inverse depth, the estimated and rendered depths cannot be directly compared. Following [29], the depth loss is instead computed using the linear correlation (Pearson correlation coefficient) between the estimated and rendered depth maps D_e and D_r:

$$\mathcal{L}_{\text{depth}} = \frac{\text{Cov}(D_e, D_r)}{\sqrt{\text{Var}(D_e)\text{Var}(D_r)}} \tag{5}$$

This depth correlation term is appended to the loss functions used in Eqs. (11) and (12).

For initializing Gaussian positions, one must first resolve the scale ambiguity of the depth estimate. This can be done by solving for a scaling σ and shift θ that fits the depth estimate to the current map. This can be modeled as a linear least squares problem of the form

$$\begin{bmatrix} \boldsymbol{d_e} & \mathbf{1} \end{bmatrix} \begin{bmatrix} \sigma \\ \theta \end{bmatrix} = \boldsymbol{d_r} \tag{6}$$

where $\boldsymbol{d_e}$ and $\boldsymbol{d_r}$ are the flattened vectors of D_e and D_r. Once the estimated depths are properly fitted to the existing map, new Gaussians can be initialized for unseen areas, initializing underoptimized maps with geometric information.

C. Inertial Fusion

Prior to optimizing the camera pose at the current frame, an initial pose estimate is required to guide optimization. A good initial estimate can lead to faster optimization times. In addition, in underoptimized areas where the convergence basins may be small, good initial estimates are essential to prevent tracking divergence.

In a monocular setting, pose estimates can be extrapolated by assuming constant velocity between consecutive frames. However, this model breaks down during presence of vigorous camera motion and low image frame rate.

Inertial measurements obtained via an IMU can be integrated to accurately propagate the camera pose between frames and yield meaningful initial estimates. Most 6-degree-of-freedom (6-DOF) IMUs provide linear acceleration measurements through accelerometers, $\mathbf{a} = [\ddot{x}, \ddot{y}, \ddot{z}]$, as well as angular velocities, $\dot{\mathbf{\Theta}} = [\dot{\alpha}, \dot{\beta}, \dot{\gamma}]$, in 3D space. Given IMU measurements are readily available in almost all consumer phones and are relatively inexpensive compared to cameras and LiDARs, this valuable information can be practically availed in any setting.

The change in position at time, t, expressed in the previous coordinate frame, ${}^{t-1}\Delta\mathbf{p_t}$, can be computed using Eq. (7),

$${}^{t-1}\Delta\mathbf{p_t} = \boldsymbol{v}_{t-1} \times t + \frac{1}{2}\mathbf{a}t^2 \tag{7}$$

where velocity is computed as $\boldsymbol{v}_{t-1} = \boldsymbol{v}_{t-2} + \boldsymbol{a}_{t-1} \times t$.

Similarly, the change in angular position at time, t, expressed in the previous coordinate frame, ${}^{t-1}\Delta\mathbf{p_t}$, can be computed using Eq. (8),

$${}^{t-1}\Delta\mathbf{\Theta_t} = \mathbf{\Theta}_{t-1} \times t \tag{8}$$

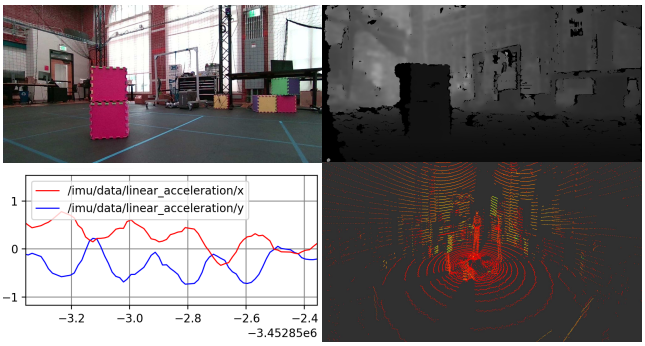

Fig. 2: Our dataset provides RGB images (top left), depth images (top right), IMU measurements (bottom left), and LIDAR point clouds (bottom right). The above examples were taken from the Ego-drive scene.

Using, Eq. (7) and Eq. (8) the relative transformation between two consecutive coordinate frames, ${}^{t-1}_{t}T_I$, can be constructed as in Eq. (9),

$$ {}^{t-1}_{t}T_I = [{}^{t-1}_{t}R \,|\, {}^{t-1}\Delta\mathbf{p_t}] \tag{9} $$

where the relative rotation matrix, ${}^{t-1}_{t}R$, is constructed using ${}^{t-1}\Delta\mathbf{\Theta_t}$ from Eq. (8). Using the static transform between the IMU and camera frame, the relative transform between consecutive camera frames can be computed as per Eq. (10),

$$ {}^{t-1}_{t}T_c = {}^{C}_{I}T \, {}^{t-1}_{t}T_I \tag{10} $$

To obtain the transform between the two arbitrary frames within a sliding window, the relative transform in Eq. (10), can be chained from the current frame to the destination frame of interest.

Note that this open-loop method does not estimate internal IMU biases. This is because errors in ${}^{t-1}_{t}T_c$ are small within short time deltas, and these small errors are optimized away by the visual camera pose optimization in Section III-D. However, this method becomes less robust as both video frame rate and IMU quality decreases. We leave closed-loop inertial fusion with bias estimation as future work.

D. Tracking

The tracking process consists of camera pose optimization given a fixed 3D Gaussian map. To enable gradient backpropagation to the camera pose, the inverse camera transformation is applied to the Gaussian map while keeping the camera fixed. This achieves identical rendering without implementing camera pose gradients in the 3DGS rasterizer. Subsequently, the 3D Gaussian map is frozen, and the camera pose is optimized according to the following loss function:

$$ \mathcal{L}_{\text{tracking}} = (\mathbb{1}_{O(\mathcal{G},T_c)})(\mathcal{L}_{\text{photo}} + \lambda_{\text{D}}\mathcal{L}_{\text{depth}}) \tag{11} $$

where $\mathbb{1}_{O(\mathcal{G},T_c)}$ is an indicator function defined as

$$ \mathbb{1}_{O(\mathcal{G},T_c)} = \begin{cases} 1 & \text{if } O(\mathcal{G},T_c) > 0.99 \\ 0 & \text{otherwise} \end{cases} $$

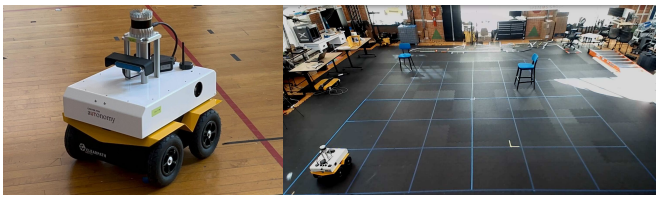

Fig. 3: A depiction of the mobile robot platform (left) equipped with a RGB-D camera, IMU, and a LiDAR and the test environment (right) featuring a 16 camera Vicon-based ground truth system.

and $\mathcal{L}_{\text{depth}}$ is the depth loss described in detail in Sec. III-B. Since the map is not guaranteed to cover the entire extent of the current frame, pixels with an opacity < 0.99 are masked.

To aid in convergence, a dynamics model can be applied prior to optimization to provide an initial guess for the camera pose. In most cases, a constant velocity model is used. However, in the presence of an inertial sensor, tracking accuracy can be improved by utilizing inertial measurements as is described in Sec. III-C. Note that tracking is skipped for the first frame as there is no existing map yet, and an identity transformation matrix is assumed as an initial guess.

E. Keyframe Selection

In a real-time setting, it is impractical to optimize the 3D Gaussian map over the entire set of video frames. However, one can exploit the temporal locality of video frames to prune away most frames from the optimization pool. This process of selectively choosing a subset of frames to optimize is known as keyframing.

In general, keyframes should be chosen to minimize the number of redundant frames processed and maximize the information gain. MM3DGS achieves this by selecting an input frame as a keyframe when the map does not contain enough information to track the current frame. This is calculated using a covisibility metric, which defines which Gaussians are visible in multiple keyframes.

To do so, first, a depth rendering is created using the current pose estimate. This depth rendering is backprojected into a point cloud. This point cloud can be projected onto consecutive image planes. We define covisibility as the percentage of points visible in the keyframe. If covisibility drops below 95%, the current frame is added as a keyframe. We empirically identified that reducing the overlap threshold leads to faster inference, but reduced rendering quality.

In the presence of visual noise, such as when an input frame depicts motion blur, keyframes with low image quality may persist and degrade reconstruction and tracking quality. To prevent such images from being selected, the Naturalness Image Quality Evaluator (NIQE) metric [1] is used to select the highest quality frame across a sliding window.

During the mapping phase, Gaussians are optimized over the set of keyframes covisible with the current frame. In this way, the optimization affects all of the relevant measurements available while minimizing processing of redundant frames and reducing computational load.

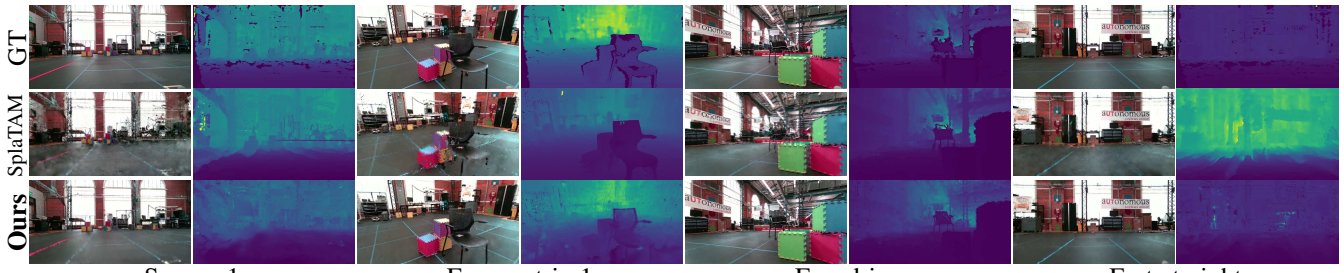

Fig. 4: Qualitative results on UT-MM dataset: RGB and depth renderings of UT-MM scenes. Note that the ground truth (GT) depths are captured with depth cameras, and thus are imperfect. Our method exhibits geometric details not present in the GT depth, as well as fewer RGB artifacts compared to SplaTAM.

TABLE I: Multi-modal SLAM results on the UT-MM dataset: ATE RMSE ↓ is in cm and PSNR ↑ is in dB, with SplaTAM is used as a baseline. Best results are in **bold**. Both depth and inertial measurements benefit tracking and image quality.

Method	Avg		Square-1		Ego-centric-1		Ego-drive		Fast-straight	
	ATE	PSNR	ATE	PSNR	ATE	PSNR	ATE	PSNR	ATE	PSNR
SplaTAM (RGB-D)	12.06	22.03	32.86	**18.67**	4.40	22.78	4.20	20.61	6.78	26.07
Ours (RGB)	46.53	19.73	73.35	16.54	5.07	23.15	70.16	17.51	37.55	21.71
Ours (RGB+IMU)	38.44	19.58	54.55	17.01	6.37	22.96	67.97	17.12	24.88	21.24
Ours (RGB-D)	7.91	23.09	21.99	17.78	1.22	24.61	**4.04**	23.58	4.40	**26.42**
Ours (RGB-D+IMU)	**4.22**	**23.19**	**8.14**	18.59	**0.97**	**24.95**	4.20	**23.61**	**3.58**	25.59

F. Gaussian Initialization

To cover unseen areas, new Gaussians are added each keyframe at pixels where the opacity is below 0.5 and the depth error exceeds 50 times the median depth error. These new Gaussians are added per-pixel, with RGB initialized at the pixel color, position initialized at the depth measurement/estimate (elaborated in Sec. III-B), opacity at 0.5, and scaling initialized isotropically to cover the extent of a single pixel at the initialized depth.

G. Mapping

The mapping process optimizes the Gaussian features visible within the current covisible set of keyframes. For the current frame and each selected keyframe, the 3D Gaussians are optimized according to

$$\mathcal{L}_{\text{mapping}} = \lambda_{\text{C}}\mathcal{L}_{\text{photo}} + \lambda_{\text{S}}\mathcal{L}_{\text{SSIM}} + \lambda_{\text{D}}\mathcal{L}_{\text{depth}} \tag{12}$$

where $\mathcal{L}_{\text{SSIM}}$ is the D-SSIM loss [30]. The other terms are identical to those in Eq. (11). To prevent unconstrained growth of Gaussians into unobserved areas, Gaussians are constrained to be isotropic.

IV. EXPERIMENTAL SETUP

A. Datasets

Several visual-inertial SLAM datasets exist to test visual-inertial fusion, however they provide grayscale images rather than RGB images, which fails to test reconstruction capabilities [31], [32]. An exception is the AR Table dataset, which contains RGB images along with inertial measurements. However, it is limited to egocentric scenes focused at only tables [33]. To bridge this gap due to the absence of RGB visual-inertial datasets, we release such a dataset, dubbed UT Multi-modal (UT-MM), captured in the Anna Hiss Gymnasium at the University of Texas at Austin. A Clearpath Jackal unmanned ground vehicle (UGV), shown in Fig. 3, is equipped with a Lord Microstrain 3DM-GX5-25 IMU, an Intel Realsense D455 camera, and an Ouster 64 line LiDAR. The UGV captures RGB and depth images at 30 frames per second, inertial measurements at 100 Hz and point clouds at 10 Hz. In addition, the ground truth trajectory of the UGV is captured using a 16 camera Vicon motion capture system with sub-mm precision. A view from one of the 16 cameras is shown in Fig. 3.

The UT-MM dataset contains eight different scenes labeled: Ego-centric-1, Ego-centric-2, Ego-drive, Square-1, Square-2, Fast-straight, Slow-straight-1, and Slow-straight-2. The three straight scenes are short scenes with the Jackal driving along a straight path with different initial positions and speeds. Square-1 and Square-2 consist of straight driving with three turns to create a loop depicting a square-shaped trajectory. Ego-drive consists of the robot being driven around several obstacles, capturing the obstacles from 360°. The Ego-centric-1 and Ego-centric-2 datasets were captured with the Jackal mounted on a omnidirectional trolley revolving around an object of interest while keeping it focused nearly at the center of the image. In some scenes, such as in the beginning of Ego-centric-1, dynamic objects are also present. These may either be cropped out or be included to test the robustness of a SLAM method to dynamic objects.

As shown in Fig. 2, there are several sensing modalities available in the UT-MM dataset. For each frame in the dataset, a corresponding RGB image and depth map is available. At the same time, IMU measurements are provided

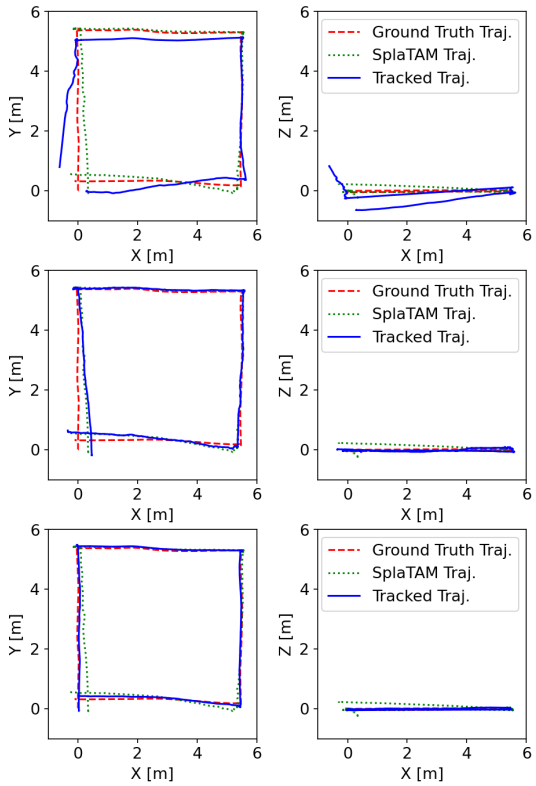

Fig. 5: Tracking results for the UT-MM Square-1 scene. The blue solid line denotes the tracked trajectory, while the red dotted line denotes the ground truth. Top: monocular RGB case exhibits substantial drift. Middle: RGB-D case fixes Z drift, but XY drift persists. Bottom: Adding IMU measurements to RGB-D fixes XY drift.

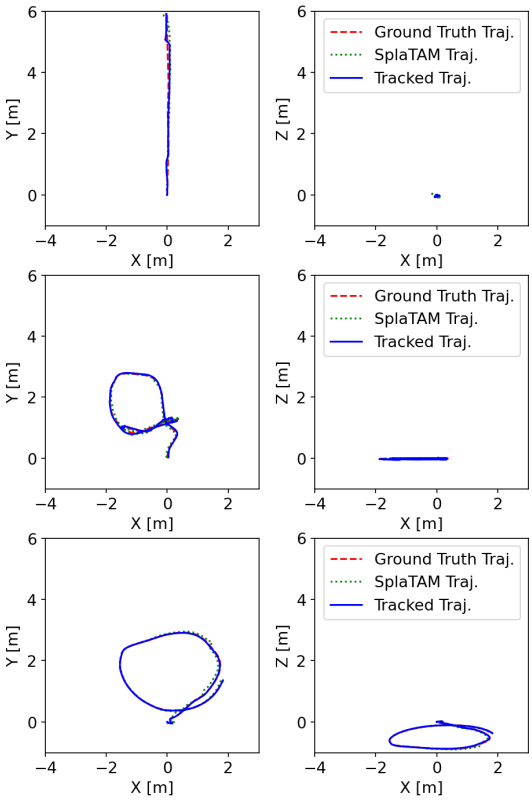

Fig. 6: RGB-D+IMU tracking trajectory results for selected scenes in the UT-MM dataset. Top: Fast-straight. Middle: Ego-drive. Bottom: Ego-centric-1.

at a rate of 100 Hz along with accurate point clouds captured. RGB-D images and IMU measurements are software-synchronized; an additional hardware synchronization board takes Pulse Per Second (PPS) input to synchronize LiDAR and IMU measurements. Our framework does not make use of highly accurate LiDAR measurements given the high cost of the sensor and lack of easy access by a common user. Nonetheless, the LiDAR can be used as ground truth during evaluation of a depth estimator.

In addition to UT-MM, we test our framework on the TUM RGB-D dataset to evaluate the performance of the monocular SLAM model in a different setting [34]. Note that TUM RGB-D provides accelerometer data but no gyroscope measurements and thus is not suitable for 6-DOF inertial fusion.

B. Metrics

For evaluation, we use two metrics: 1) tracking accuracy, measured via absolute tracking error root mean square error (ATE RMSE), and 2) scene reconstruction quality, measured via peak signal-to-noise ratio (PSNR). An Umeyama point alignment algorithm aligns the trajectory generated using the model of interest with ground truth, treating the camera frame as fixed. [35]. SplaTAM, an RGB-D 3DGS SLAM model that this work extends, is used as a baseline [11].

C. Implementation

We run our framework on an RTX A5000 GPU with 24GB VRAM. We do not multi-thread our framework and hence the tracking and mapping threads run sequentially. For all scenes, we run pose optimization for 100 iterations followed by 150 iterations of map optimization. We set the loss function hyperparameters to $\lambda_C = 0.8$, $\lambda_S = 0.2$, and $\lambda_D = 0.001$ or 0.05 for the RGB-D and RGB models, respectively.

V. RESULTS AND DISCUSSION

We conduct a comprehensive qualitative and quantitative evaluation of our framework on the UT-MM dataset and perform an ablation study on the TUM RGB-D dataset to justify the use of NIQE keyframing and Pearson correlation loss.

A. Effect of Incorporating Multi-modal Sensor Information

Table I shows the tracking and rendering results for SplaTAM, a state-of-the-art RGB-D 3DGS baseline, and various configurations of MM3DGS on scenes from the UT-MM dataset. Considering the average ATE and PSNR metrics, it is evident that addition of inertial measurements improves tracking and image quality for both the RGB and RGB-D configurations. The RGB-D+IMU configuration consistently outperforms others, demonstrating a nearly **3x**

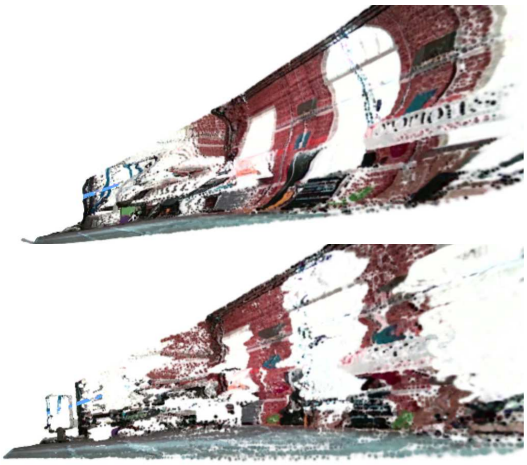

Fig. 7: Top: depth initialized by DPT. Bottom: depth initialized by depth camera. The DPT estimate exhibits warping along the Z axis compared to the depth camera.

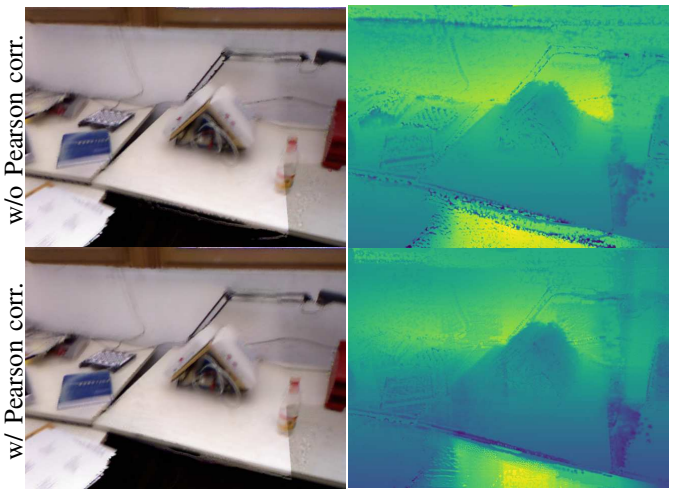

Fig. 8: RGB and depth renderings of TUM RGB-D with and without Pearson correlation loss. The addition of Pearson correlation results in similar RGB quality while increasing geometric consistency, particularly along edges. Note the shift in brightness on the right side of the RGB renders; this is due to a sudden exposure change in the input camera frames.

improvement in ATE RMSE and **5%** improvement in PSNR compared to the baseline on average. This demonstrates the value of integrating inertial measurements in our framework enhancing trajectory tracking and rendering quality.

Furthermore, we showcase qualitative comparisons of the rendered RGB and depth images against SplaTAM and ground truth in Fig. 4. The superior image rendering quality of our framework on all of the scenes compared to SplaTAM signifies the value of the RGB-D+IMU configuration of MM3DGS. Due to the use of lightly weighted depth correlation loss rather than a heavily weighted direct depth loss implemented in SplaTAM, MM3DGS is able to capture geometric details that are not present in the noisy depth input through the photometric loss in Eq. (4). For instance, in the Ego-centric-1 scene, our depth rendering includes the many small holes on the back of the chair while SplaTAM instead overfits to the depth input and is not able to capture the holes. MM3DGS also exhibits significantly fewer visual artifacts in its RGB renderings compared to SplaTAM which depicts considerably more missing colors and floaters. Both methods render at **90 fps** on an RTX A5000 GPU.

To visualize the effect of incorporating IMU and depth measurements on tracking, we plot the trajectory of the RGB, RGB-D, and RGB-D+IMU configurations on the UT-MM Square-1 scene as shown in Fig. 5. The integration of depth measurements helps with trajectory alignment by eliminating drift in the Z-axis. The dense depth estimates provided by DPT are indeed warped in the Z direction as shown in Fig. 7. However, drift along the XY plane persists and is addressed only with the addition of inertial measurements in our framework. This highlights the limitations of monocular depth estimators and the importance of integrating both inertial and depth sensing modalities.

B. Performance of the Monocular RGB Configuration

To further gauge the performance of the proposed monocular RGB SLAM configuration, we evaluate its performance on the TUM RGB-D dataset. The ATE RMSE results on select scenes from the dataset are shown in Table II. In ego-centric scenes, the RGB model exhibits comparable performance to SplaTAM's RGB-D model, demonstrating that depth measurements do not provide much added information given a wide range of multi-view constraints. However, the RGB model fails on scenes that involve longer trajectories which may be alleviated by the addition of loop closure methods.

TABLE II: Monocular RGB configuration results on the TUM RGB-D dataset. ATE RMSE ↓ is in cm. Monocular RGB SLAM provides comparable performance as RGB-D baselines.

Method	fr1/desk	fr1/desk2	fr2/xyz
SplaTAM (RGB-D)	**3.35**	6.54	**1.24**
Ours (RGB)	4.17	**5.53**	2.02

TABLE III: Monocular RGB configuration ablation results on the TUM RGB-D `freiburg1/desk2` scene. ATE RMSE ↓ is in cm and PSNR ↑ is in dB. Performance is best with both Pearson Corr. loss and NIQE keyframing.

NIQE Keyframing	Pearson Corr. Loss	ATE RMSE	PSNR
✓	✗	Fails	Fails
✗	✓	15.33	19.67
✓	✓	**5.53**	**20.56**

C. Ablation Study

We also perform an ablation study on the `freiburg1/desk2` scene due to presence of high motion blur and exposure changes. Table III showcases

RGB tracking results with Pearson correlation depth loss and NIQE keyframing ablated. Without the Pearson correlation depth loss, tracking fails. The effect of the depth correlation loss in improving geometric consistency is illustrated in Fig. 8. Further, the addition of NIQE keyframing increases both tracking accuracy and image quality.

VI. CONCLUSIONS

We presented MM3DGS, a multi-modal SLAM framework built on a 3D Gaussian map representation that utilizes visual, inertial, and depth measurements to enable real-time photorealistic rendering and improved trajectory tracking. We evaluate our framework on a new multi-modal dataset, UT-MM, that includes RGB-D images, IMU measurements, LiDAR depth, and ground truth trajectories. MM3DGS achieves superior tracking accuracy and rendering quality compared to state-of-the-art baselines. In addition, we present an ablation study to highlight the importance of our framework. MM3DGS can be implemented in a wide range of applications in robotics, augmented reality, and mobile computing due to its use of commonly available and inexpensive sensors. As future work, MM3DGS can be extended to include tightly-coupled IMU fusion and loop closure to further enhance tracking performance.

REFERENCES

[1] A. Mittal, R. Soundararajan, and A. C. Bovik, "Making a "completely blind" image quality analyzer," *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013.

[2] M. Contreras, N. P. Bhatt, and E. Hashemi, "A stereo visual odometry framework with augmented perception for dynamic urban environments," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2023, pp. 4094–4099.

[3] J. Polvi, T. Taketomi, G. Yamamoto, A. Dey, C. Sandor, and H. Kato, "SlidAR: A 3d positioning method for SLAM-based handheld augmented reality," *Computers & Graphics*, vol. 55, pp. 33–43, 2016.

[4] M. Contreras, N. P. Bhatt, and E. Hashemi, "DynaNav-SVO: Dynamic Stereo Visual Odometry With Semantic-Aware Perception for Autonomous Navigation," *IEEE Transactions on Intelligent Vehicles*, 2024.

[5] H. Bavle, P. De La Puente, J. P. How, and P. Campoy, "VPS-SLAM: Visual planar semantic SLAM for aerial robotic systems," *IEEE Access*, vol. 8, pp. 60 704–60 718, 2020.

[6] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: a versatile and accurate monocular SLAM system," *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[7] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual–inertial odometry using nonlinear optimization," *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.

[8] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6229–6238.

[9] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 786–12 796.

[10] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian Splatting SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[11] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, track & map 3d gaussians for dense RGB-D SLAM," *arXiv*, 2023.

[12] D. Xu, H. Liang, N. P. Bhatt, H. Hu, H. Liang, K. N. Plataniotis, and Z. Wang, "Comp4d: Llm-guided compositional 4d scene generation," *arXiv preprint arXiv:2403.16993*, 2024.

[13] S. Hong, J. He, X. Zheng, H. Wang, H. Fang, K. Liu, C. Zheng, and S. Shen, "LIV-GaussMap: LiDAR-inertial-visual fusion for real-time 3d radiance field map rendering," *arXiv preprint arXiv:2401.14857*, 2024.

[14] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "DTAM: Dense tracking and mapping in real-time," in *2011 international conference on computer vision*. IEEE, 2011, pp. 2320–2327.

[15] T. Schops, T. Sattler, and M. Pollefeys, "BAD SLAM: Bundle adjusted direct RGB-D SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 134–144.

[16] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt, "Bundlefusion: Real-time globally consistent 3d reconstruction using on-the-fly surface reintegration," *ACM Transactions on Graphics (ToG)*, vol. 36, no. 4, p. 1, 2017.

[17] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.

[18] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5501–5510.

[19] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics (ToG)*, vol. 41, no. 4, pp. 1–15, 2022.

[20] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (ToG)*, vol. 42, no. 4, pp. 1–14, 2023.

[21] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[22] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, "Pixelwise view selection for unstructured multi-view stereo," in *European Conference on Computer Vision (ECCV)*, 2016.

[23] Y. Fu, S. Liu, A. Kulkarni, J. Kautz, A. A. Efros, and X. Wang, "Colmap-free 3d gaussian splatting," *arXiv preprint arXiv:2312.07504*, 2023.

[24] C. Yan, D. Qu, D. Wang, D. Xu, Z. Wang, B. Zhao, and X. Li, "GS-SLAM: Dense visual SLAM with 3d gaussian splatting," 2024.

[25] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense SLAM with gaussian splatting," 2023.

[26] J. Jeong, T. S. Yoon, and J. B. Park, "Towards a meaningful 3d map using a 3d lidar and a camera," *Sensors*, vol. 18, no. 8, p. 2571, 2018.

[27] C. Jiang, D. P. Paudel, Y. Fougerolle, D. Fofi, and C. Demonceaux, "Static-map and dynamic object reconstruction in outdoor scenes using 3-d motion segmentation," *IEEE Robotics and Automation Letters*, vol. 1, no. 1, pp. 324–331, 2016.

[28] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," *ArXiv preprint*, 2021.

[29] Z. Zhu, Z. Fan, Y. Jiang, and Z. Wang, "FSGS: Real-time few-shot view synthesis using gaussian splatting," 2023.

[30] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[31] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The EuRoC micro aerial vehicle datasets," *The International Journal of Robotics Research*, 2016. [Online]. Available: http://ijr.sagepub.com/content/early/2016/01/21/0278364915620033.abstract

[32] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stuckler, and D. Cremers, "The TUM VI benchmark for evaluating visual-inertial odometry," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2018. [Online]. Available: http://dx.doi.org/10.1109/IROS.2018.8593419

[33] C. Chen, P. Geneva, Y. Peng, W. Lee, and G. Huang, "Monocular visual-inertial odometry with planar regularities," in *Proc. of the IEEE International Conference on Robotics and Automation*, London, UK, 2023. [Online]. Available: https://github.com/rpng/ov_plane

[34] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. of the International Conference on Intelligent Robot Systems (IROS)*, Oct. 2012.

[35] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, vol. 13, no. 04, pp. 376–380, 1991.